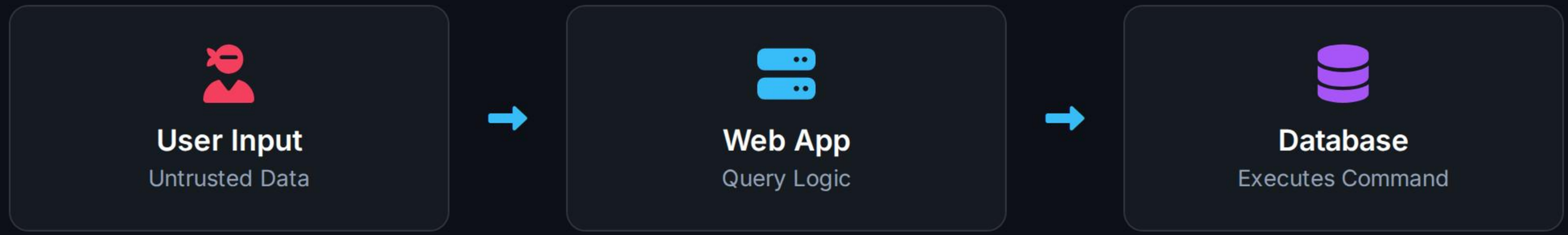
Can You Change What the Database Is Asking?

SQL Injection — When Data Becomes Code

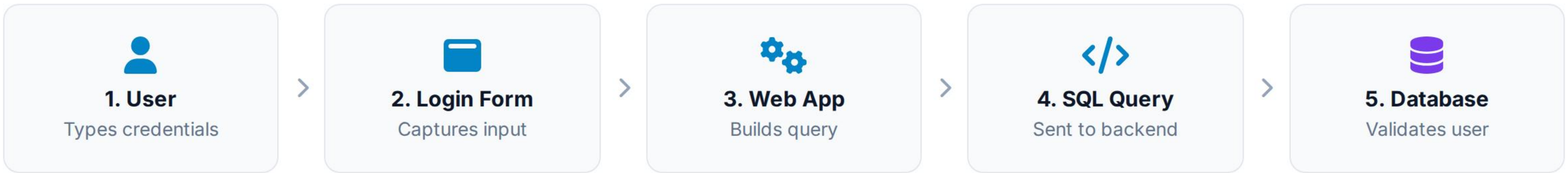


SPEAKER NOTES & INTERACTIVITY

- **Opening Hook:** Ask audience: "What if typing text into a box could rewrite the software?"
- **Key Concept:** Computers treat code and data differently. SQLi happens when data tricks the computer into running it as code.

THE BASICS

What Happens When You Log In?



Audience Question: Where along this path do you think an attacker can interfere?

SPEAKER NOTES & INTERACTIVITY

- **Interactive Moment:** Pause at the question box. Let 2-3 participants guess.
- **Key Takeaway:** Attackers interfere at step 1 (the form), but the vulnerability occurs at step 3 when constructing step 4.

UNDER THE HOOD

The Normal SQL Query

```
SELECT * FROM users WHERE username = 'alice' AND password = '1234';
```

SQL Code (Fixed Command)

Instructions predefined by software developers: `SELECT`, `WHERE`, `AND`.

User Data (Variable Input)

Supplied directly by end users: `'alice'` and `'1234'`.

> **Golden Rule: Safe applications strictly isolate SQL Command Structure from User-Controlled Data.**

SPEAKER NOTES & INTERACTIVITY

- **Explanation:** Explain SQL simply—it's just asking a table questions. "Give me records matching user X and password Y."
- **Focus:** Emphasize the quotes around single values. Single quotes delimit text string boundaries in SQL.

THE ROOT CAUSE

Where Things Go Wrong

VULNERABLE CODE PATTERN

```
query = "SELECT * FROM users WHERE username = '" + userInput + "'"
```

String Concatenation Flaw

The app glues user input directly into the SQL string before sending it to the database driver.

→

Boundary Collapse

Untrusted input crosses the boundary and becomes active executable database code.

SPEAKER NOTES & INTERACTIVITY

- **Analogy:** Imagine giving a robot a note that says: "Eat lunch, then erase your own memory." If it reads text as orders, it breaks.
- **Key Point:** The database cannot tell where developer code ends and user input begins.

EXPLOITATION

The Injection Payload

ATTACKER ENTERS IN USERNAME BOX:

```
' OR '1'='1' #
```

NORMAL INTENDED QUERY

```
WHERE username = 'alice'
```

INJECTED MODIFIED QUERY

```
WHERE username = '' OR '1'='1' # '
```

Audience Check: Is '1' equal to '1'? Since TRUE OR ANYTHING = TRUE, the whole check passes!

SPEAKER NOTES & INTERACTIVITY

- **Step-by-Step Breakdown:** 1. First single quote breaks out of string. 2. OR '1'='1' forces query to TRUE. 3. # comments out trailing password check.
- **Audience Question:** "Is '1' equal to '1'?" → Instant always-true condition bypasses authentication!

CORE INSIGHT

What Actually Happened?

1

Attacker Input

Injects SQL syntax characters

2

Escapes String

Quote breaks out of data mode

3

Adds SQL Logic

Appends new commands

4

Modified Query

Database executes altered query

”The attacker didn’t break the database.
They changed the question being asked to the database.”

SPEAKER NOTES & INTERACTIVITY

- **Key Takeaway:** Emphasize this slide—it is the central conceptual takeaway of the entire webinar.
- **Mental Model:** The database worked perfectly according to instructions. The problem was the instructions were altered!

IMPACT & CONSEQUENCES

What Can SQL Injection Cause?

Auth Bypass
Log in as administrator without credentials

Data Access
Extract sensitive customer & user tables

Data Tampering
Modify balances or alter database records

Data Deletion
Drop entire tables or compromise servers

Note: Total impact depends on application structure and database account privileges.

SPEAKER NOTES & INTERACTIVITY

- **Real-world Context:** SQLi isn't just seeing other people's names—it can destroy databases completely.
- **CIA Triad Alignment:** Confidentiality (Read data), Integrity (Modify data), Availability (Delete data).

CASE STUDY

Real-World Case: TalkTalk Breach (2015)

OCTOBER 2015

Vulnerable Web Page

Legacy web page contained basic SQL injection vulnerabilities.

EXPLOITATION

156,000+ Records

Attackers accessed customer personal data and financial details.

AFTERMATH

£77 Million Impact

Fines, customer churn, and severe reputational damage followed.

Takeaway: A simple web vulnerability on a forgotten page can lead to catastrophic organizational breaches.

SPEAKER NOTES & INTERACTIVITY

- **Context:** The attackers were teenagers using basic automated tools against an unpatched site.
- **Lesson:** You don't need nation-state tools to cause major damage if basic inputs are unvalidated.

PRACTICAL DEMO

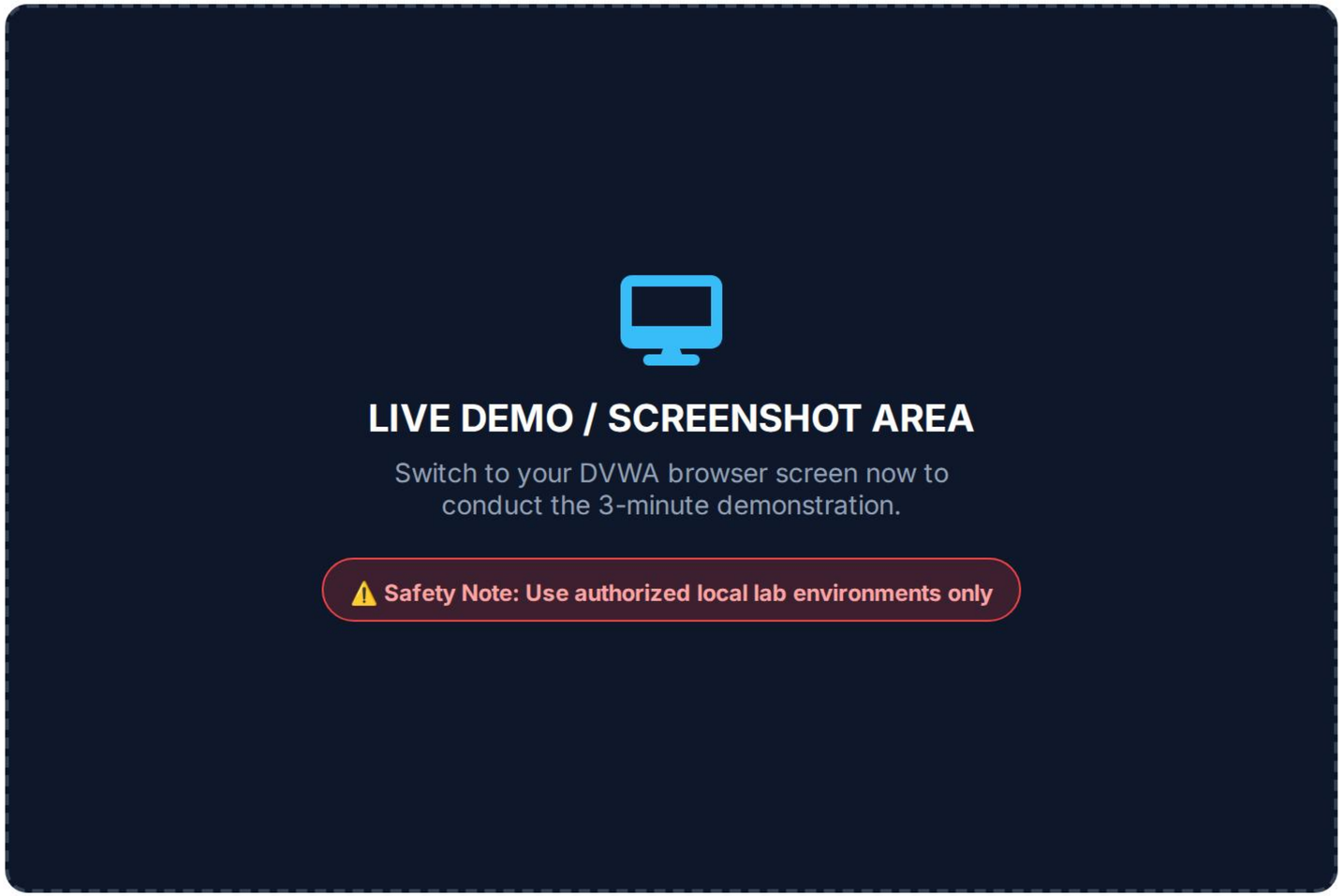
Live Practical: DVWA Lab

1. Open DVWA Lab
Set Security Level = LOW

2. Test Normal Input
Submit ID: 1 → Observe user result

3. Inject SQL Payload
Submit: `' OR '1'='1`

4. Compare Output
Observe full database dump



SPEAKER NOTES & INTERACTIVITY

- **Live Demo Transition:** "Now let's see this in action inside DVWA."
- **Key Steps during Demo:** Show single quote error first, then show full payload dumping all accounts.

INTERACTIVE QUIZ

Audience Challenge

SCENARIO: ATTACKER INPUT

A user enters `admin' --` into the username field.

```
SELECT * FROM users WHERE username = 'admin' --' AND password = 'secret';
```

What will the database see when evaluating this query?
(Think about what the '--' comment character does to the rest of the line!)

SPEAKER NOTES & INTERACTIVITY

- **15-Second Pause:** Give the audience time to type answers in chat or raise hands.
- **Reveal Answer:** The database sees `SELECT * FROM users WHERE username = 'admin'`. The password check is ignored completely!

REMEDIATION

How Do We Stop SQL Injection?

❌ Unsafe: Concatenation ❌

Glues raw strings together into executable code.

✅ Safe: Parameterized Query ✅

Uses placeholders so inputs are treated purely as data.

SQL CODE ≠ USER DATA

Prepared Statements
Pre-compiles query structure in database engine.

Input Validation
Enforces strict allowlists for user data types.

Least Privilege
Restricts database account permissions.

SPEAKER NOTES & INTERACTIVITY

- **Key Concept:** Parameterized queries pre-compile the SQL structure before user input ever touches it.
- **Takeaway:** Even if input contains `' OR '1'='1`, it is evaluated literal string value, not SQL commands.

SQL Injection = Untrusted Input Changing SQL Logic

UNSAFE: User Input → Web App → Unsafe Query Concatenation → Database

SAFE: User Input → Web App → Parameterized Query → Database

"If you were the lead developer, where in your stack would you stop the attack?"

SPEAKER NOTES & INTERACTIVITY

- **Final Q&A:** Open the floor to audience questions.
- **Closing Words:** Thank everyone for attending and encourage practicing safely in DVWA or PortSwigger Web Security Academy!